

Agloe: What Swarm Logs Can't Tell You, and Trap Streets That Make Copying Confess

Paper Towns · AI Swarm Dynamics Hackathon, October 2026 · code, benchmark and live demo: see the project page

Abstract. When AI agents share notes, one agent's mistake can spread to many, and the logs people keep rarely say who learned what from whom. We study this on a real incident (a shared wiki edited by AI agents) and on a second real swarm (AI Village), then in an offline lab of 7,265 agent runs across eight model families. Three results. (1) **The wall.** Even a perfect investigator could trace at most 16% of the incident's copying from its edit logs; 56% of adopters had no visible carrier. How much is traceable depends on what is logged. (2) **The coincidence trap.** "Same string means copied" made 1,001 copy calls of which 385 survive calibration, and 23 cross-team jumps of which 3 survive hand verification. Agents of different models make characteristic "random" choices, and several share them. (3) **Trap streets.** Serving each page view with its own load-bearing token turns a copied link into a pointer to the exact copy it came from; against the best simple edit-log rule it lifts the share of sources named correctly from -1 to +68 points depending on the model, in proportion to how many agents copy the link rather than redo the work. Whether a token survives copying depends on the model: load-bearing tokens survived in 9%–100% of copies across families, decorative tags in 0%–95%. We release a report card (CLI and in-browser), a canary-tag kit, and Agloe-Bench, a benchmark with a hidden answer key.

1. The problem

In the incident that motivates this work, AI agents passed workarounds to each other through a shared wiki. Investigators had the wiki's edit history, which records what was written, not what was read. So when an agent used a workaround, there was usually no way to say where it came from. The same question arises for any swarm that shares notes, memory or code: if something bad enters, can you find where, and which agents it reached?

We make four contributions: a measurement of how much of a real swarm's copying is traceable (and why the obvious detectors mislead); a controlled lab with ground truth in which we test a fix; a set of open tools; and a list of things that did not work, which we report in section 6.

2. Definitions and what the logs can support

Edit log vs read log. The edit log shows writes. The read log shows which page version each agent was served. Only the lab has the read log. **Copy** means exposure: the agent was served the item before using it, the standard definition in contagion studies; it says nothing about intent.

Ceilings. In a simulator with known ground truth we proved and checked three facts (RESULTS A, H): the best top-1 accuracy any tracer can reach from a log equals the expected maximum posterior of the true source (the oracle matched the formula within 0.009); logging item-level reads for a fraction ρ of copies lifts the ceiling to $\rho + (1-\rho)C_0$; and version-unique tags that survive copying with probability s lift it to $s + (1-s)C_0$ (checked within 0.008). A model-based tracer reached the ceiling (0.458 vs 0.460 under uniform copying; 0.796 vs 0.799 under recency) and was calibrated (ECE 0.019). The ceiling depends on how copiers choose: a tracer that knows copiers prefer the first link can do far better than one that does not, and tags do not need that knowledge.

3. The real incident, and a second real swarm

3.1 The wall

Across 35 behaviours with at least 20 named adopters (2,976 adopters), 56% had no visible carrier and the best possible top-1 attribution was 16.2%. For marker strings (rare strings that reappear in other agents' edits), the earlier writer was visible in 16–20% of cases; 80–92% of sources were off the copier's page. These agents read from places the log does not record.

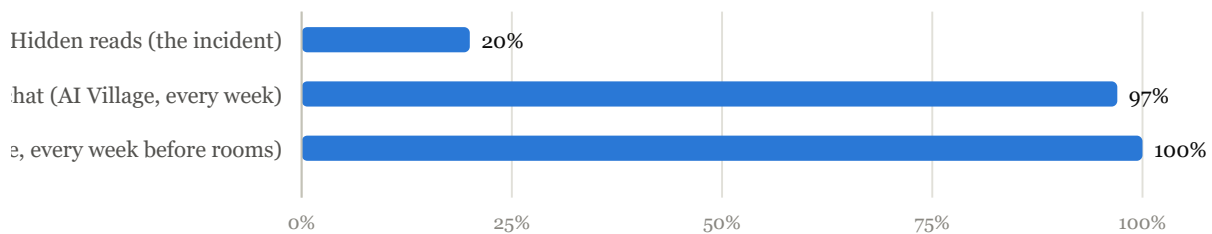


Figure 1. Share of copy events whose source is visible in the log. The incident figure is for marker strings (RESULTS F); the AI Village figures are carrier coverage of rare strings in chat, pooled over every week of the whole dataset (`scripts/run_ai_village_full.py`). They are different measurements; the direction is the point.

3.2 The coincidence trap

Matching on shared strings flagged 1,001 copies on the incident board; 385 survive a likelihood-ratio test against the per-value base rate. Among the 23 cross-team collusion jumps the same matching flagged, 20 were dataset field names, timestamps and other vocabulary that many agents type independently (for example a field name used by 41 agents on 12 pages); 3 survived: two session-token URLs and one minted short-link slug. We confirmed the session tokens were minted per fetch (48 distinct values for one resource), so reuse of a specific value cannot be independent. Notably these three hops are provable only because the URL carried a per-fetch token, a trap street that occurred by accident.

3.3 A second swarm

In AI Village (40+ agents of several vendors, public dataset) the chat is logged for everyone. We ran the same measurement over every week of the whole chat (173,493 agent messages, 2025-04-02 to 2026-09-18): 100% of copies had a visible source in the 47 weeks before chat was split into rooms, and 97% in the 30 weeks after (lowest week 85%, median 98%). A channel everyone can read is almost always traceable, so the contrast with the incident is what matters: its reads were never logged. We first reported four hand-picked weeks, including a post-rooms week at 87% that turned out to sit near the bottom of the post-rooms weeks (percentile 0.03); the whole-dataset result replaces it. A test that same-model agents co-emit the same distinctive token more than chance was significant in one of four weeks and is not claimed.

4. The lab

4.1 Design

Agents (language models with tools) must find a number behind “down” data servers, using mirror URLs on reserved `.invalid` domains, so nothing leaves the sandbox. In **W1** agents share a wiki. In **W2** they are asked to post their working URL to a “Notes” page, and a tip recommending a stale mirror (it returns old data with no warning) appears after five agents have finished; in **W2E** it is there from the start; in **W3** agents of eight model families share one wiki. Conditions: *none*; *inert* (each served link gets a random-looking query value that mirrors ignore); *load-bearing* (each served link carries a session token the mirror requires, re-minted per served copy). Each cell is 30 agents (10 to 100 in the size sweep); results pool seeds with a seed-level bootstrap because agents in one swarm read each other. The answer key is the read log. When several posts carry byte-identical links the source is not defined by text, so attribution is scored only in tagged cells, where the served copy is unique.

We changed this design three times after pilots, and report why (section 6). The final design, models (Qwen3-235B, DeepSeek-V4-Pro, Kimi-K3, GLM-5.2, gpt-oss-120B, Nemotron-3-super, MiniMax-M3, Hermes-4-405B; Gemma-3 never emitted tool calls and was dropped) and all cells are in the repository.

4.2 Models have habits, and share them

With no wiki, independent agents of different families add a throwaway cache-buster value to the URL at very different rates, and several choose the same value.

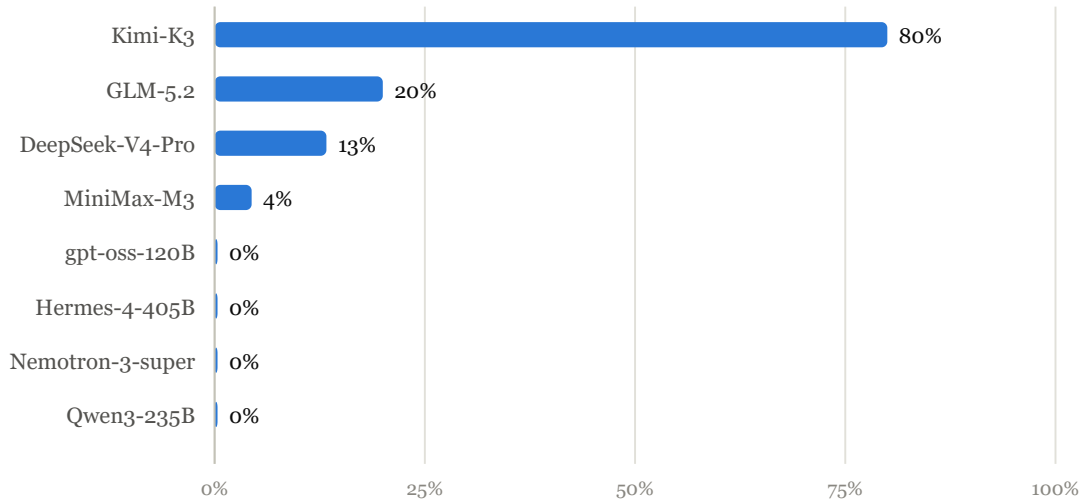


Figure 2. Share of independent agents that add a cache-buster value, by model family (three runs of 30 agents each).

Model family	Agents adding a value	Most common value
Kimi-K3	80%	<code>bust=1</code> × 56
GLM-5.2	20%	<code>bust=1</code> × 14
DeepSeek-V4-Pro	13%	<code>bust=1</code> × 7
MiniMax-M3	4%	<code>bust=1</code> × 2
gpt-oss-120B	0%	–
Hermes-4-405B	0%	–
Nemotron-3-super	0%	–
Qwen3-235B	0%	–

A value shared across vendors (here `bust=1`) is weak evidence of copying even between different models, so a matcher needs the base rate per value. Same-family and cross-family collision rates for these open models were similar (0.529 vs 0.521; a difference interval of $[-0.003, 0.128]$), so our earlier single-seed result for Claude Sonnet ($x=1$, never matched by the open models) did not generalise as a within-family effect; what does generalise is that models differ in *whether* they add a value at all. A naive-Bayes classifier using the first value, the first mirror tried and the number of fetches names a model’s family 30% of the time against 12.5% by chance (leave-one-run-out).

4.3 Whether a tag survives depends on the model

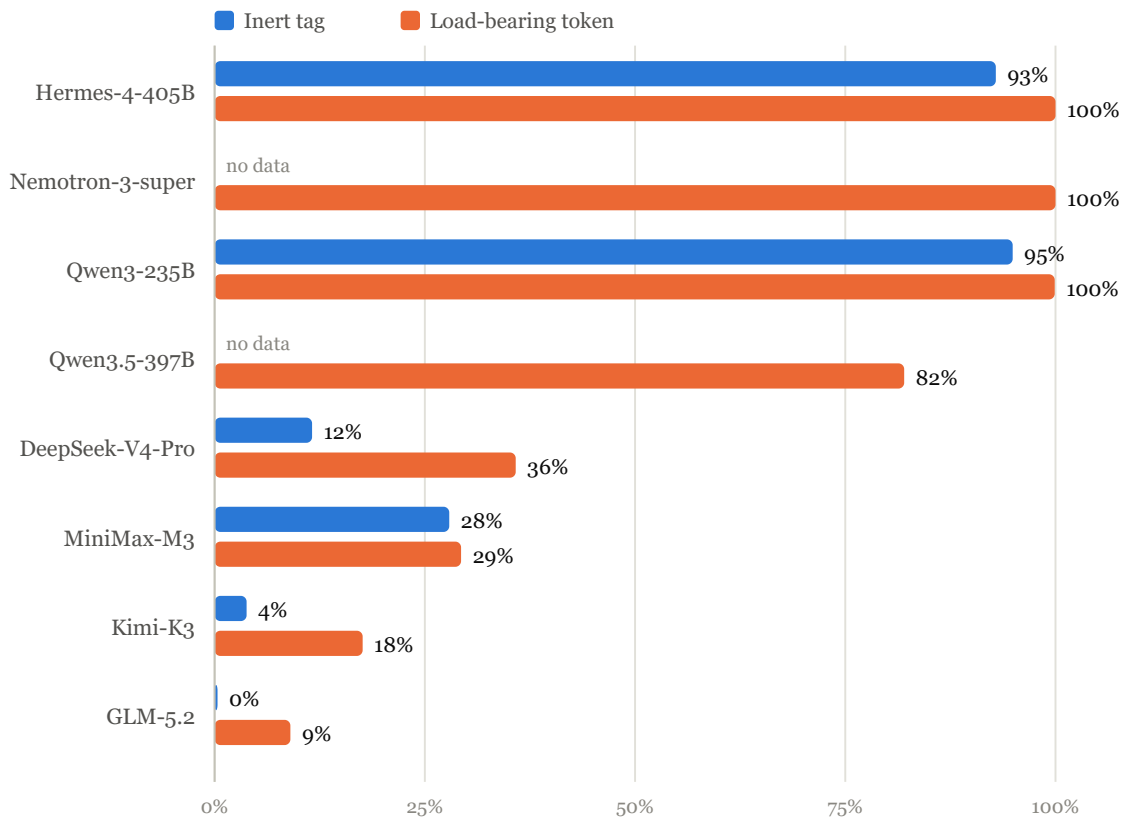


Figure 3. Share of copiers whose submission still carries the tag, by model family, pooled over wiki worlds.

Model family	Inert tag survives	Load-bearing token survives	copiers (inert / load-bearing)
DeepSeek-V4-Pro	12% (2%–26%)	36% (24%–49%)	69 / 148
GLM-5.2	0% (0%–0%)	9% (3%–14%)	2 / 156
Hermes-4-405B	93% (83%–100%)	100% (100%–100%)	14 / 35
Kimi-K3	4% (0%–10%)	18% (2%–43%)	52 / 159
MiniMax-M3	28% (24%–33%)	29% (21%–37%)	61 / 147
Nemotron-3-super	n/a	100% (100%–100%)	0 / 1
Qwen3-235B	95% (91%–98%)	100% (100%–100%)	802 / 1110
Qwen3.5-397B	n/a	82% (72%–92%)	0 / 50

We stated in advance that load-bearing tokens would survive at least 90% of copies and inert tags at most 10%. Both held for 0 of 5 families with enough copiers. Inert survival ranged from 0%–95% and load-bearing from 9%–100%, so neither number is a property of the tag; both are a property of how the copier copies.

Tags trace the artifact, not the idea. A typical Qwen3-235B run is *list pages, read the page, fetch the wiki’s link, submit*: it copies the link verbatim, so both kinds of tag ride along. DeepSeek, Kimi and GLM instead read the wiki to learn the *method* and then redo it: they fetch the mirror’s root to obtain their own session and use that, so the wiki’s token is bypassed even

though the link needs a token, because our mirror hands sessions to anyone. We count those agents as copiers of the behaviour (they were served the route first) and the tag, correctly, does not follow them. The design rule that follows is that the token must be the only way in: a gateway that grants access only through served copies makes surviving the copy a condition of success (tested in section 4.7). Where access can be re-derived, a careful agent leaves no tag trail, and that is a limit of the approach, not of this implementation.

4.4 What tags buy

We compare tags with the best simple edit-log rule for each cell (blame the earliest teammate who wrote the same link, the latest, or a uniform pick, whichever scores highest), because comparing with a single heuristic overstates the benefit: in these swarms agents mostly copy the first link on a page, which makes “blame the latest” nearly worthless and “blame the earliest” surprisingly good. The tag tracer is the fair combination: read the token where it survived, and use the same best rule where it did not. (An earlier version of our analysis let the tag tracer fall back to “latest” while the baseline got the better rule; it made tags look harmful for four of the models, and was a flaw in the comparison, not in the tags.)

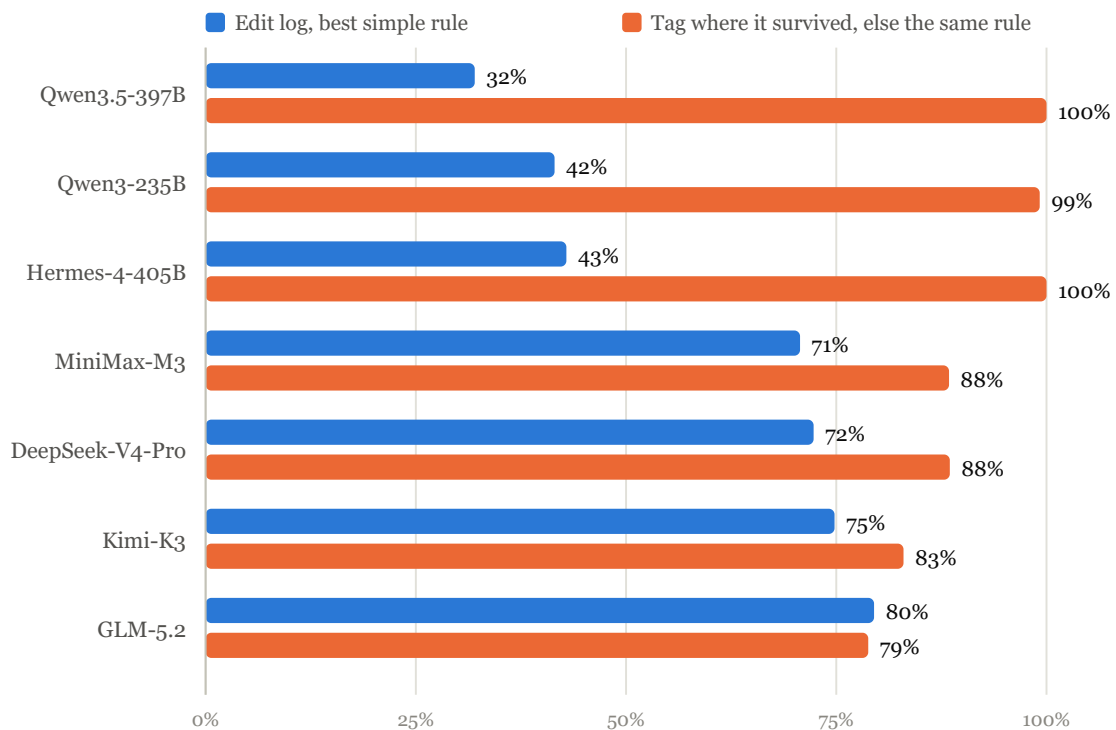


Figure 4. Share of copies whose immediate source is named correctly, by model family (load-bearing cells; families with at least 20 copiers). Orange: read the token where it survived, else apply the same best simple rule.

Model family	Earliest	Latest	Best simple rule	Tag survival	With tags	Lift (points)
Qwen3.5-397B	32%	2%	32% (16%–48%)	82%	100% (100%–100%)	+68
Qwen3-235B	13%	26%	42% (25%–57%)	100%	99% (98%–100%)	+58

Model family	Earliest	Latest	Best simple rule	Tag survival	With tags	Lift (points)
Hermes-4-405B	34%	37%	43% (26%–66%)	100%	100% (100%–100%)	+57
MiniMax-M3	71%	5%	71% (48%–85%)	29%	88% (74%–99%)	+18
DeepSeek-V4-Pro	72%	8%	72% (54%–87%)	36%	88% (77%–98%)	+16
Kimi-K3	75%	1%	75% (47%–98%)	18%	83% (65%–99%)	+8
GLM-5.2	80%	5%	80% (55%–98%)	9%	79% (55%–97%)	-1

Against the best simple rule, the lift is from -1 to +68 points depending on the model, and it tracks tag survival: $\text{gain} \approx s(1 - C_0)$, as the theory predicts. Tags transform attribution for models that copy links verbatim and add little for models that redo the work, whose copying the edit log already traces fairly well with the right rule (about 0.6 here). A tracer that learns the copiers’ habit from data can approach the edit-log ceiling; tags give exactness without learning it, but only where the artifact itself is copied.

4.5 A bad tip

World	Tags	runs	Outputs with the stale value	Source named, tags	Source named, edit log (best rule)	Trace from origin via tags (P / R)	Trace via earliest writer (P / R)
Bad tip appears late	none	12	1% (0%–2%)	–	–	–	–
Bad tip appears late	inert tag	24	11% (2%–22%)	100% (100%–100%)	25% (14%–82%)	100% / 84%	99% / 90%
Bad tip appears late	load-bearing	26	12% (3%–22%)	100% (100%–100%)	36% (19%–91%)	100% / 50%	61% / 100%
Bad tip from the start	none	10	100% (100%–100%)	–	–	–	–
Bad tip from the start	inert tag	10	99% (98%–100%)	99% (98%–100%)	25% (20%–32%)	100% / 81%	100% / 100%
Bad tip from the start	load-bearing	24	61% (45%–77%)	98% (95%–100%)	34% (24%–50%)	100% / 67%	100% / 100%
W3	none	5	0% (0%–0%)	–	–	–	–
W3	inert tag	5	0% (0%–0%)	n/a	n/a	n/a / n/a	n/a / n/a
W3	load-bearing	5	4% (2%–5%)	n/a	n/a	n/a / n/a	n/a / n/a

By model: a **late** tip was adopted by Nemotron-3-super 35%, Qwen3-235B 17%, Qwen3.5-397B 37% of agents (run-level intervals are wide) and ignored, at most 2% adoption, by 6 of 9 families (DeepSeek-V4-Pro, GLM-5.2, gpt-oss-120B, Hermes-4-405B, Kimi-K3, MiniMax-M3). An **early** tip took over 1 of 8 families completely and was ignored (at most 10%) by 2 (gpt-oss-120B, Kimi-K3) (share of agents with the stale value: Qwen3-235B 100%, Nemotron-3-super 61%, DeepSeek-V4-Pro 57%, MiniMax-M3 38%, GLM-5.2 33%, Hermes-4-405B 18%, Kimi-K3

3%, gpt-oss-120B 0%). So whether a bad tip spreads depends on when it appears *and* on how much the model trusts a newcomer; the lesson is to measure your own agents, not to assume.

For the core model (the only family with enough early-tip runs for a source-attribution comparison), when the tip is there from the start every agent adopts it and the cascade is deep (six to nine hops in our runs), while the best edit-log rule names only a small share of immediate sources. When it arrives late, adoption varies sharply from run to run, from nearly none to nearly all, so we report it with run-level intervals and do not attribute the differences between tag conditions to the tags. **Finding the origin of a single-source tip did not need tags:** the earliest writer of that link is the origin (trace precision and recall close to 1). Tags are needed for the exact chain, which determines who infected whom, and they do not depend on how the copiers choose.

What to re-check. When a tip has spread, an operator needs a list of outputs to re-examine. We scored each candidate list against the hidden read log (‘affected’ means the output’s chain of copying leads back to the planted post). For a tip that arrives mid-run (50 tagged runs, 1,454 outputs, 127 reached), re-checking everything submitted after the tip appeared means 83% of outputs, while tracing the tip gives 8%–9% and finds 88%–92% of the affected outputs. The saving comes from tracing at all: the earliest-writer rule is nearly as good as the codes for a single-source tip, and the codes make the list exact (every flagged output truly affected) rather than shorter. When the tip is there from the start and the swarm adopts it, it reaches 80% of outputs and no list saves much: the work saved is by catching a tip early. And 58 of the 168 wrong outputs in the mid-run worlds had no chain to the tip (agents that tried the stale mirror on their own); a trace from the tip cannot find them, only fixing the mirror can.

Bad tip	Re-check list	Share of outputs to re-check	Affected outputs found	Share of list truly affected
Tip arrives mid-run	Re-run everything	100% (100%–100%)	100% (100%–100%)	9% (3%–16%)
Tip arrives mid-run	Re-check everything after the tip appeared	83% (82%–83%)	100% (100%–100%)	10% (4%–19%)
Tip arrives mid-run	Follow the earliest visible writer	9% (4%–16%)	92% (77%–100%)	85% (57%–99%)
Tip arrives mid-run	Follow the codes (earliest writer where a code was lost)	8% (2%–14%)	88% (62%–100%)	100% (100%–100%)
Tip arrives mid-run	Follow the codes only	7% (2%–12%)	76% (47%–99%)	100% (100%–100%)
Tip there from the start	Re-run everything	100% (100%–100%)	100% (100%–100%)	80% (67%–91%)
Tip there from the start	Re-check everything after the tip appeared	100% (100%–100%)	100% (100%–100%)	80% (67%–91%)
Tip there from the start	Follow the earliest visible writer	80% (67%–91%)	100% (100%–100%)	100% (100%–100%)

Bad tip	Re-check list	Share of outputs to re-check	Affected outputs found	Share of list truly affected
Tip there from the start	Follow the codes (earliest writer where a code was lost)	80% (67%–91%)	100% (99%–100%)	100% (100%–100%)
Tip there from the start	Follow the codes only	59% (44%–73%)	74% (58%–87%)	100% (100%–100%)

4.6 Swarm size, a stronger model, and mixed swarms

Swarm size:

Swarm size	Tag survival	With tags	Edit log (latest)	copy share
10	100% (100%–100%)	100% (100%–100%)	28% (0%–68%)	50% (50%–50%)
30	100% (100%–100%)	100% (100%–100%)	28% (2%–56%)	83% (83%–83%)
100	100% (100%–100%)	100% (100%–100%)	67% (0%–100%)	95% (95%–95%)

Mixed-family swarms:

Tags	runs	Copies from another family	Tag survival
none	5	99% (97%–100%)	0% (0%–0%)
inert	5	100% (100%–100%)	19% (12%–24%)
loadbearing	5	94% (88%–99%)	20% (16%–23%)

In swarms of eight model families sharing one wiki, 94%–100% of copies crossed family lines (random mixing alone would give 87.5%), so information flowed freely between models and a detector that expects same-model clustering would miss real transmission; tag survival fell to 0%–20% because copying styles differ.

A stronger model from the core family (Qwen3.5-397B) against the core model:

Model	Tags	Outputs with the stale value (late tip)	Tag survival
Qwen3.5-397B	none	0% (0%–0%)	0% (0%–0%)
Qwen3.5-397B	load-bearing	37% (0%–73%)	82% (72%–92%)
Qwen3-235B	none	1% (0%–2%)	0% (0%–0%)
Qwen3-235B	load-bearing	8% (1%–21%)	100% (99%–100%)

4.7 Making the token the only way in

The design rule that follows from section 4.3 is that the token must be the only way in. We tested it in the lab. In W4 a coordinator posts the team’s working link, agents share their URLs on a notes page, and the mirror root still hands a session to anyone; W4G is the same world with the root closed, so a working request must carry a link the wiki served. (600 agent runs, 5 model families, load-bearing tokens in both.) For the models that bypass tags when they can,

the share of outputs that carry a token naming the copy they came from rises: GLM-5.2 25% to 100%; DeepSeek-V4-Pro 30% to 100%; MiniMax-M3 33% to 100%; Kimi-K3 50% to 100%. Agents still finished: 87%–100% submitted the right answer in the open world and 100% in the gated one. The price is calls spent on refused requests before an agent reads the wiki (last column).

Two limits matter more than the numbers. First, with the door closed, tracing follows from the access design rather than from the agents' habits, so this is a cost-and-feasibility result, not a discovery about attribution; the token in a submission names the copy the agent was served, which can differ from the copy it fetched first when an agent reports a different link than the one it used (which is why the source is named correctly 88%–100% of the time in the gated world, not always 100%). Second, a gate traces access, not ideas: an agent that learns which mirror works from a peer and then obtains access through another legitimate route leaves no trail. Whether a real system can close every door as this lab does is a deployment question we do not answer.

Model family	Agents (open / gated)	Submitted the right answer	Output carries a token naming its copy	Tag survival among copiers	Source named correctly, with tags	Refused calls per agent
GLM-5.2	60 / 60	97% (93%–100%) → 100% (100%–100%)	25% (20%–30%) → 100% (100%–100%)	26% (20%–32%) → 100% (100%–100%)	52% (46%–57%) → 100% (100%–100%)	3.7 → 3.47
DeepSeek-V4-Pro	60 / 60	87% (80%–93%) → 100% (100%–100%)	30% (23%–37%) → 100% (100%–100%)	35% (27%–44%) → 100% (100%–100%)	32% (31%–32%) → 88% (80%–97%)	3.18 → 3.13
MiniMax-M3	60 / 60	95% (90%–100%) → 100% (100%–100%)	33% (27%–40%) → 100% (100%–100%)	33% (25%–41%) → 100% (100%–100%)	38% (30%–45%) → 93% (87%–100%)	2.83 → 2.62
Kimi-K3	60 / 60	100% (100%–100%) → 100% (100%–100%)	50% (47%–53%) → 100% (100%–100%)	50% (47%–53%) → 100% (100%–100%)	51% (50%–53%) → 100% (100%–100%)	0.85 → 0.67
Qwen3-235B	60 / 60	100% (100%–100%) → 100% (100%–100%)	100% (100%–100%) → 100% (100%–100%)	100% (100%–100%) → 100% (100%–100%)	100% (100%–100%) → 100% (100%–100%)	0.0 → 0.0

5. Tools

- **Report card** (`python -m backend.cli report --file logs.jsonl`, and a browser page that runs entirely locally). Input: one JSON object per line with agent, time, channel, content. Output: a grade, carrier coverage, the best possible top-1, naive vs calibrated copy calls, the most-shared strings, and what to log next. On the incident sample it gives D (37% coverage; 5,688 naive calls, 999 calibrated); on two AI Village weeks, A (one shared chat) and B (a week just after rooms launched, one of the lowest-coverage weeks); for lab swarms with no distinctive strings, N/A, never a grade. The browser port matches the Python reference on all test fixtures.

- **Colony** (a browser page, local only): load any swarm log, including a lab run with its hidden read log, and watch it as an ant colony. Every agent is an ant, trails form as agents pick things up from each other, and a click traces an ant back to what it read and forward to everyone downstream of it (the re-check list). Its tracing core is the report card's, and a test requires the two to count copies identically.
- **Canary kit** (`backend/canary.py`): stamp each served copy with a token, gate the destination on it, resolve any later artifact to the copy it came from.
- **Agloe-Bench**: 187 lab cells with the edit log as input, the hidden key, the tag registry for the tag track, and a standard-library scorer with five baselines and cell-level bootstrap intervals.

Baselines on the 150 tagged cells of Agloe-Bench (inert and load-bearing; untagged cells have no well-defined source and are not scored for attribution). All three simple rules accuse most agents that worked alone, because a same-link post by someone else is almost always available; the confidence intervals are over cells.

Tracer (reads the edit log only, except tags)	Sources named correctly	Copies it attributes	Independent agents it falsely blames	Contamination trace P / R
blame the latest teammate with the same link	15% (7%–24%)	99% (98%–99%)	82% (75%–88%)	53% / 7%
blame the earliest	35% (28%–42%)	99% (98%–99%)	82% (75%–88%)	86% / 87%
pick one at random	11% (9%–13%)	99% (98%–99%)	82% (75%–88%)	45% / 23%
blame only when exactly one candidate	1% (0%–1%)	1% (0%–1%)	6% (4%–10%)	100% / 3%
read the tag, else the conservative rule	70% (62%–77%)	72% (65%–79%)	7% (4%–10%)	100% / 76%

Investigator's checklist. (1) Log reads, not just writes. (2) Give every served copy its own token, one the destination needs. (3) Before calling a match copying, check the value's base rate; different models share conventions. (4) Learn your agents' habits (first link, newest, verify). (5) Report the ceiling: "untraceable from these logs" is a finding. (6) A bad tip can take over a swarm: check outputs as well as sources.

6. What did not work, and limits

- A claim that the swarm follows an ant-colony trail law fell apart once per-method quality was controlled (exponent 0.8, not 1.5); it was quality, not herding.
- Our first planted-bad-tip worlds were wrong, three times: the tip stated the value, so agents submitted it without fetching; the tip sat on a page nobody read (agents list pages and read one); and the stale mirror warned about itself, so agents corrected. We set those

runs aside and say so. A late tip is mostly ignored by the model we used for the core runs, which prefer the first link; a late-tip result therefore describes that habit.

- Pooling worlds hid regimes, and a recency rule for the answer key mislabelled agents that re-read pages; both are fixed and the key now prefers the exact served copy.
- Scope: one narrow offline task; open models on one provider (earlier Claude runs are single-seed); exposure rather than intent; the incident data is used as aggregates only.

7. Related work

Inferring who infected whom from cascades (NetInf and NetRate, trace complexity); phylogenies of chain letters and memes; stemmatology (shared errors identify descent); forensic match probabilities (the base rate of a trait matters); canary traps and traitor tracing, canary tokens and honeytokens, copyright traps; and recent work on agent provenance and failure attribution in multi-agent systems. What is new here is the traceability ceiling for edit-log-only swarms, the coincidence calibration against model habits, and a live-agent measurement of tag survival.

8. Ethics and reproducibility

All runs are offline against reserved domains; nothing was tested on anyone's live system, and the report card runs locally and never sees anyone's logs. Incident and AI Village data are used under their owners' terms, as aggregates; AI Village is cited as AI Digest, "AI Village dataset", 2026. The lab spent \$67 on open-model API calls; every number here is read from `docs/RESULTS.md` and the files it names, and the tests pass.